elements. If we set the x position of each rectangle based on the `i` value (multiplied by the width of the rectangle), then we get a step closer to a bar chart:

```
d3.select("svg")
  .selectAll("rect")
  .data([15, 50, 22, 8, 100, 10])
  .enter()
  .append("rect")
  .attr("width", 10)
  .attr("height", function(d) {return d;})
  .style("fill", "blue")
  .style("stroke", "red")
  .style("stroke-width", "1px")
  .style("opacity", .25)
  .attr("x", function(d,i) {return i * 10});
```

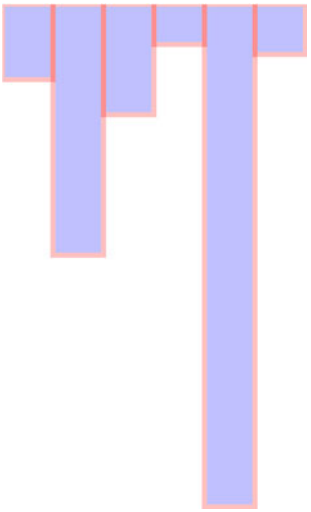


Figure 2.14 SVG rectangles are drawn from top to bottom.

Our histogram seems to be drawn from top to bottom, as seen in figure 2.14, because SVG draws rectangles down and to the right from the 0,0 point that we specify. To adjust this, we need to move each rectangle so that its y position corresponds to a position that is offset based on its height. We know that the tallest rectangle will be 100. The y position is measured based on the distance from the top left of the canvas, so if we set the y attribute of each rectangle equal to its length minus 100, then the histogram is drawn in the manner we'd expect, as shown in figure 2.15.

```
d3.select("svg")
  .selectAll("rect")
  .data([15, 50, 22, 8, 100, 10])
  .enter()
  .append("rect")
  .attr("width", 10)
  .attr("height", function(d) {return d;})
  .style("fill", "blue")
  .style("stroke", "red")
  .style("stroke-width", "1px")
  .style("opacity", .25)
  .attr("x", function(d,i) {return i * 10;})
  .attr("y", function(d) {return 100 - d;});
```

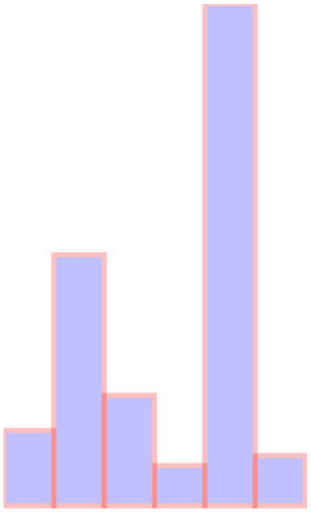


Figure 2.15 When we set the y position of the rectangle to the desired y position minus the height of the rectangle, the rectangle is drawn from bottom to top from that y position.

2.2.3 Integrating scales

This way of building a chart works fine if you're dealing with an array of values that correspond directly to the height of the rectangles relative to the height and width of your `<svg>` element. But if you have real data, then it tends to have widely divergent values that don't correspond directly to the size of the shape you want to draw. The previous code doesn't deal with an array of values like this:

```
[14, 68, 24500, 430, 19, 1000, 5555]
```